

PROJECT ARGUS

An AI-Powered Smart Glasses System for the Visually Impaired

Final Machine Learning Pipeline and Execution Plan

Two-Speed Vision-Language Architecture with Gemma 4 E2B and YOLO-World

Revision incorporating Phase-0 feasibility findings

Deployment platform: NVIDIA Jetson Orin Nano Super (8 GB)

Perception: 2 × Arducam AR0234 stereo pair · 1 × Arducam IMX477P wide camera

Contents

1. Executive Summary
2. Phase-0 Feasibility Findings and Resulting Decision
3. Revised Component Stack
4. The Two-Speed Architecture (Updated)
5. YOLO-World as the Grounding Tool
6. Compute Resources and Operating Rules
7. Implementation — Notebook by Notebook (with code)
8. On-Device Deployment and Memory Budget
9. Phased Execution Roadmap
10. Risk Register
11. Summary of Changes

1. Executive Summary

This document finalises the machine learning pipeline for Project ARGUS following a formal Phase-0 feasibility investigation into the on-device deployment of the two candidate foundation models. The investigation confirmed that **Google Gemma 4 E2B** is well supported on the NVIDIA Jetson Orin Nano Super and runs with multimodal (image) input via a native **llama.cpp** build. It also established that **NVIDIA LocateAnything-3B cannot presently be deployed on this device**: the model ships in BF16 only, exposes no INT4 quantisation or GGUF/TensorRT export path, relies on a custom Parallel Box Decoding head whose acceleration kernel is restricted to Hopper and Blackwell GPUs, and is explicitly documented by NVIDIA as targeting datacentre GPUs and the larger Jetson Thor, not the Orin Nano.

In response, the grounding role is reassigned from LocateAnything-3B to **YOLO-World**, an open-vocabulary detector that runs natively under TensorRT on the Orin Nano at interactive speed. This substitution preserves the Two-Speed architecture in full: the reasoning agent (Gemma 4 E2B) continues to orchestrate a called grounding tool, but the tool is now one that the target hardware can actually execute, and at far lower latency than LocateAnything would have delivered even if it had been deployable.

Final Locked Decisions

Architecture: Two-Speed Vision-Language design (non-ML safety reflex + event-driven VLM agent). Unchanged.

Reasoning agent: Google Gemma 4 E2B, multimodal, via native llama.cpp. Confirmed deployable.

Grounding tool: YOLO-World (open-vocabulary), via TensorRT. Replaces LocateAnything-3B.

Safety reflex loop: non-ML, rule-based, continuously running. Unchanged.

Privacy gate: mandatory face/text blur before any frame reaches the agent. Unchanged.

2. Phase-0 Feasibility Findings and Resulting Decision

The previous version of this plan identified the on-device deployment of LocateAnything-3B as the single largest technical risk and designated its resolution as a project-gating activity. That investigation has now been completed; its findings are summarised here.

2.1 Gemma 4 E2B — Confirmed deployable

- **Vendor support.** Both NVIDIA and Google published Day-0 support for Gemma 4 E2B on the Jetson Orin Nano, including a working reference agent combining speech-to-text, the Gemma 4 vision model with camera input, and text-to-speech, running entirely on a single 8 GB Orin Nano Super.
- **Inference path.** The model is deployed via a native llama.cpp build using a quantised GGUF checkpoint together with the official vision projector file, exposing an OpenAI-compatible endpoint that accepts image input.
- **Measured performance.** Approximately 11 tokens per second decode at INT4 in the device's high-power mode, with a resident footprint of roughly 3 to 4.5 GB — comfortably within budget.

2.2 LocateAnything-3B — Not presently deployable on Orin Nano

Obstacle	Detail
Weights are BF16 only	No INT4 build and no published quantisation recipe exist; the checkpoint alone approaches the device's usable memory.
No edge inference path	NVIDIA's model card states TensorRT, TensorRT-LLM and Triton are not yet supported; only PyTorch/transformers is available.
Custom decoder	Parallel Box Decoding uses Python-level hybrid control flow that cannot be represented in GGUF and has no ONNX or TensorRT export route.
Acceleration kernel restricted	The MagiAttention kernel underpinning its speed is Hopper/Blackwell only; on the Orin Nano's Ampere GPU it falls back to slow PyTorch attention.
Not a listed target	NVIDIA lists supported hardware as A100/Hopper/Lovelace/Blackwell and Jetson Thor; the Orin Nano is not included.

Even under the most favourable assumptions, a single grounding query on the Orin Nano would have required several seconds to tens of seconds, well beyond the interactivity threshold appropriate for an assistive wearable. The model remains an option only on a larger device (Jetson Orin NX 16 GB, AGX Orin, or Thor).

Decision

Adopt YOLO-World as the grounding tool. It provides open-vocabulary detection (the user may name any object), runs natively under TensorRT on the Orin Nano at approximately 160 milliseconds per query, and leaves ample memory for the resident reasoning agent. The Two-Speed architecture and all other components are retained unchanged.

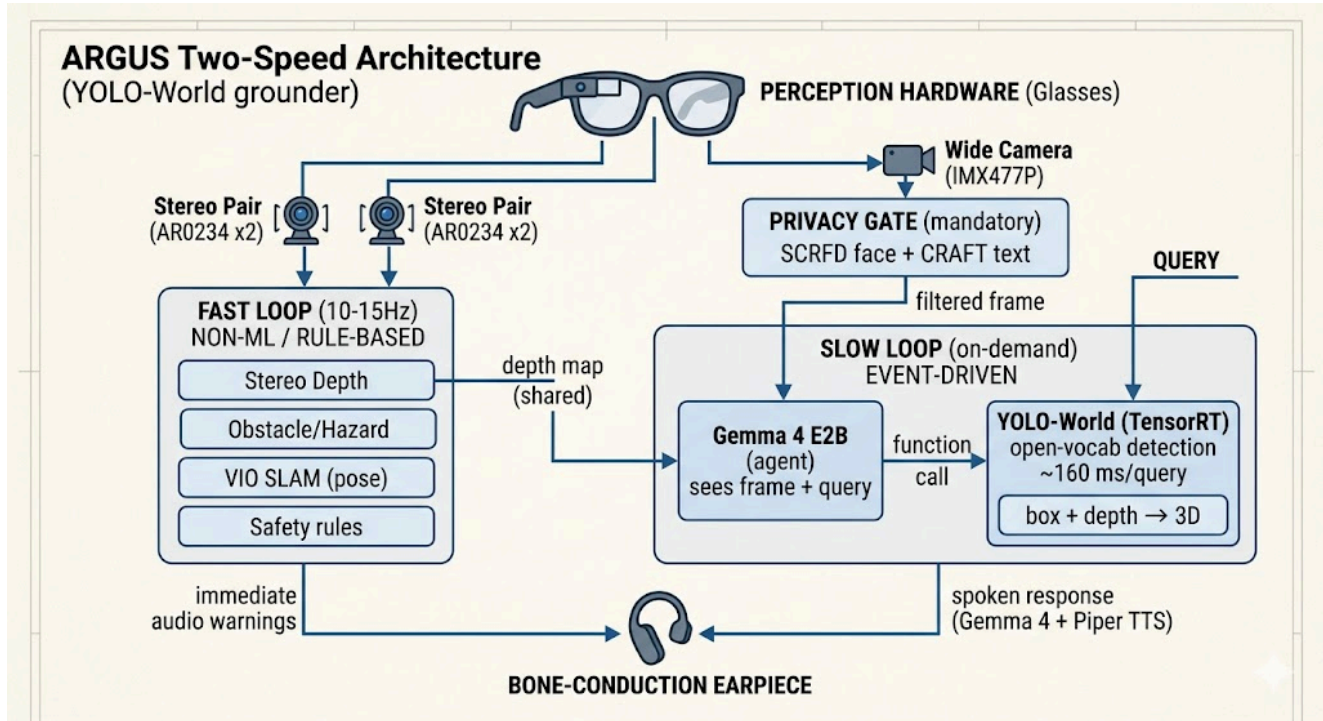
3. Revised Component Stack

Role	Model / Method	Loop	Deployment	Trained?
Reasoning agent	Gemma 4 E2B (multimodal)	Slow	llama.cpp (GGUF, INT4)	Optional LoRA
Grounding tool	YOLO-World	Slow	TensorRT (INT8/FP16)	Light fine-tune
Stereo depth	RAFT-Stereo / SGBM	Fast	TensorRT	Retained
Obstacle & hazard	Geometric rules	Fast	Native code	No
SLAM	OpenVINS → ORB-SLAM3	Fast	Native code	No
Privacy gate	SCRFD + CRAFT	Slow	TensorRT	No (pretrained)
Wake word	openWakeWord	—	CPU	Brief training
Speech-to-text	faster-whisper (tiny)	—	CPU (INT8)	No
Text-to-speech	Piper	—	CPU	No

4. The Two-Speed Architecture (Updated)

The architecture is unchanged in principle; only the grounding tool invoked by the agent differs. A continuously running, non-machine-learning *fast loop* guarantees the user's physical safety, while an event-driven *slow loop* provides intelligent interaction on demand.

4.1 Information flow



4.2 Worked interaction: “find my keys”

1. The fast loop is already running, tracking obstacles and pose continuously and independently.
2. The user speaks; the wake word triggers and faster-whisper transcribes “find my keys”.
3. The wide-camera frame passes through the privacy gate (faces blurred) and is presented to Gemma 4 E2B together with the query.
4. Gemma 4 recognises an object-localisation intent and issues a function call to YOLO-World with the open-vocabulary class “keys”.
5. YOLO-World returns a bounding box in roughly 160 ms; the box centre is sampled against the depth map to obtain a 3D position.
6. Gemma 4 converts the position into guidance and Piper speaks it: “Your keys are on the table to your right, about one metre ahead.”

5. YOLO-World as the Grounding Tool

YOLO-World is an open-vocabulary object detector that accepts a set of text class names at inference time and detects the corresponding objects, without retraining for new categories. This open-vocabulary property is the essential requirement inherited from the LocateAnything design: the user must be able to ask for any object by

name, not only a fixed list. Unlike a 3-billion-parameter vision-language grounder, YOLO-World is a compact detector that exports cleanly to ONNX and TensorRT and runs at interactive speed on the Orin Nano.

5.1 Why YOLO-World is the correct substitute

- **Open vocabulary.** Detects arbitrary, user-named objects via text prompts, preserving the natural-language flexibility the project requires.
- **Edge-deployable.** Standard ONNX and TensorRT export paths exist; it runs on the Orin Nano's Ampere GPU without exotic kernels.
- **Fast.** Roughly 160 ms per query under TensorRT, against several seconds for a 3B VLM grounder — an order-of-magnitude improvement that materially benefits the user.
- **Light.** A few hundred megabytes resident, allowing it to coexist with the resident Gemma 4 agent within the 8 GB budget.

5.2 NanoOWL as an alternative

NanoOWL, an NVIDIA TensorRT optimisation of OWL-ViT, is a viable alternative open-vocabulary detector also designed for the Jetson platform. YOLO-World is recommended as the primary choice for its higher throughput and mature [Ultralytics](#) tooling, with NanoOWL retained as a fallback should prompt-level flexibility prove insufficient.

6. Compute Resources and Operating Rules

Training requirements for the revised pipeline are modest, since both principal models are used largely as released. The following resources and operating rules govern all development and training work. They are derived from established project practice and are to be applied without exception.

6.1 Resource allocation

Resource	Specification	Role in the project
GTX 1060 (local)	6 GB VRAM, ~150 GB free disk	Code authoring, debugging, data preparation, stereo calibration, ONNX validation
Colab T4	15 GB	First-pass testing; export, quantisation, and inference-only validation
Colab L4	22.5 GB	Mid-tier fine-tuning where the T4 is memory-constrained
Colab A100	40 GB	Reserved for any heavier fine-tuning; used only after a one-epoch test passes on a T4
Google Drive	Persistent	Storage of model artefacts and dataset archives only

6.2 GPU selection rule

GPU selection policy

```

Writing / debugging      -> Local GTX 1060 or Colab CPU (no cost)
First test of a new notebook -> T4
Routine inference / export -> T4
T4 out-of-memory         -> L4 (cheaper than A100)

```

Heavier fine-tune -> A100, only after a 1-epoch T4 profiling pass

6.3 Non-negotiable efficiency rules

- **Never train or read datasets directly from Google Drive.** Copy archives to local session storage first, then extract and read from there.
- **Always use mixed precision** for any training or fine-tuning run.
- **Profile one epoch on a low-cost accelerator** before scaling to a higher-cost one. GPU utilisation below 80 per cent indicates a data-pipeline problem to be fixed before scaling.
- **Checkpoint to persistent storage at regular intervals**, never only at the end of a run.
- **Release the accelerator immediately** on completion of a run; an idle high-cost accelerator wastes compute budget.
- **Apply early stopping**; do not train to completion out of habit when validation loss has plateaued.

6.4 Platform constraints established during Phase 0

- The **llama.cpp** Python bindings will not build on the Colab Python 3.12 environment; language-model inference work is therefore performed on the Jetson, not in Colab.
- Audio libraries such as **pyaudio** will not build on Colab; all audio handling is performed on the Jetson.
- **TensorRT engines are device-specific** and must be built on the Jetson itself; they cannot be cross-compiled in Colab. Colab is used to produce ONNX, which is then converted to a TensorRT engine on the device.
- All ONNX exports use **dynamo=False**, as the PyTorch 2.x dynamo exporter has been observed to fail on certain vision operators.

7. Implementation — Notebook by Notebook (with code)

The implementation is organised as a sequence of Google Colab notebooks for the cloud-side work (export, quantisation, light fine-tuning) and a set of on-device scripts for the Jetson-side work (TensorRT engine building, inference, integration). Each block below is a self-contained cell.

Notebook 00 — Environment and Drive setup

Cell 00-1 | GPU check and Drive mount

```
from google.colab import drive
import torch, subprocess
print(subprocess.run(['nvidia-smi'], capture_output=True, text=True).stdout)
print("GPU :", torch.cuda.get_device_name(0))
print("CUDA:", torch.cuda.is_available())
drive.mount('/content/drive')
```

Cell 00-2 | Project folders on Drive (artefacts only)

```
import os
BASE = '/content/drive/MyDrive/ARGUS'
for d in ['models/grounding', 'models/depth', 'models/privacy',
         'models/speech', 'exports/onnx', 'datasets', 'logs']:
    os.makedirs(f'{BASE}/{d}', exist_ok=True)
print("Project tree ready at", BASE)
```

Notebook 01 — YOLO-World: validate, fine-tune, export

This notebook validates open-vocabulary detection, optionally fine-tunes on indoor objects relevant to the user, and exports to ONNX for on-device TensorRT conversion. Per the efficiency rules, datasets are staged to local session storage, not read from Drive.

Cell 01-1 | Install and load YOLO-World

```
!pip install -q ultralytics
from ultralytics import YOLOWorld

# Pretrained open-vocabulary detector (no training needed to start)
model = YOLOWorld('yolov8s-worldv2.pt')
print("YOLO-World loaded.")
```

Cell 01-2 | Open-vocabulary detection test

```
# Set arbitrary user-named classes at inference time
model.set_classes(["keys", "coffee mug", "remote control",
                  "white cane", "medicine bottle", "door handle"])

results = model.predict('/content/test_indoor.jpg', conf=0.25)
for box in results[0].boxes:
    cls = results[0].names[int(box.cls)]
    xxyy = box.xyxy[0].tolist()
    print(f"{cls:16s}  conf={float(box.conf):.2f}  box={[round(v) for v in xxyy]}")
```

Cell 01-3 | (Optional) fine-tune on indoor objects

```
# Stage dataset locally first (never read training data from Drive)
!cp /content/drive/MyDrive/ARGUS/datasets/indoor.zip /content/
!unzip -q /content/indoor.zip -d /content/indoor

# Fine-tune with mixed precision; checkpoint to Drive periodically
model.train(
    data='/content/indoor/data.yaml',
    epochs=40, imgsz=640, batch=16,
    amp=True,                               # mixed precision (efficiency rule)
    device=0,
    project='/content/drive/MyDrive/ARGUS/logs',
    name='yoloworld_indoor',
    save_period=10,                          # periodic checkpointing
    patience=5)                             # early stopping
```

Cell 01-4 | Export to ONNX (dynamo=False, per Phase-0 rule)

```
# Export ONNX on Colab; the TensorRT engine is built later ON the Jetson
model.export(
    format='onnx',
    imgsz=640,
    opset=17,
    simplify=True,
    dynamo=False)                          # required: dynamo exporter is unreliable here

import shutil
shutil.copy('yolov8s-worldv2.onnx',
            '/content/drive/MyDrive/ARGUS/exports/onnx/yoloworld_640.onnx')
print("ONNX exported to Drive.")
```

Notebook 02 — Privacy gate (pretrained, export only)

Cell 02-1 | Face and text detectors

```
!pip install -q insightface onnxruntime-gpu
from insightface.app import FaceAnalysis
face = FaceAnalysis(name='buffalo_s',
                  providers=['CUDAExecutionProvider', 'CPUExecutionProvider'])
face.prepare(ctx_id=0, det_size=(640,640))
print("SCRFD face detector ready (pretrained, no training).")
# CRAFT text-region weights are downloaded for the Jetson side separately.
```

Cell 02-2 | Blur function (the mandatory gate)

```
import cv2
```



```
def privacy_gate(frame_bgr):
    """Blur all faces BEFORE the frame is shown to the agent."""
    faces = face.get(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
    out = frame_bgr.copy()
    for f in faces:
        x1,y1,x2,y2 = [int(v) for v in f.bbox]
        roi = out[max(0,y1):y2, max(0,x1):x2]
        if roi.size:
            out[max(0,y1):y2, max(0,x1):x2] = cv2.GaussianBlur(roi,(51,51),0)
    return out
# The agent must NEVER receive a frame that has not passed through this gate.
```

Notebook 03 — Gemma 4 E2B prompt and tool contract (design + dry run)

Because llama.cpp does not build on Colab, the Gemma 4 agent is run on the Jetson. This notebook is used only to design and dry-run the system prompt and the function-calling contract, using the hosted API as a stand-in for behaviour design.

Cell 03-1 | ARGUS system prompt

```
ARGUS_SYSTEM = """You are ARGUS, an assistant embedded in smart glasses for a
blind user. You see one camera frame and hear one question. Rules:
1. Answer in one or two short spoken sentences.
2. State any safety hazard first.
3. To locate a specific named object, call the tool find_object(name).
4. Never describe faces or private documents.
Return either a spoken answer, or a single tool call."""
```

Cell 03-2 | Tool contract for YOLO-World

```
# The agent emits this JSON; the Jetson runtime executes YOLO-World and
# returns boxes, which are fused with depth into a 3D position.
TOOL_SPEC = {
    "name": "find_object",
    "description": "Locate a named object in the current camera view.",
    "parameters": {
        "type": "object",
        "properties": {
            "name": {"type": "string",
                    "description": "open-vocabulary object name, e.g. 'keys'"}
        },
        "required": ["name"]
    }
}
```

Notebook 04 — Speech components (validation)

Cell 04-1 | Whisper STT (INT8) sanity check

```
!pip install -q faster-whisper
from faster_whisper import WhisperModel
stt = WhisperModel('tiny', device='cpu', compute_type='int8')
print("faster-whisper tiny INT8 ready (runs on Jetson CPU).")
```

The wake word is trained with [openWakeWord](#) from roughly 200 short recordings of the hotword, and [Piper](#) provides text-to-speech. Both run on the Jetson CPU and require no GPU work.

Notebook 05 — On-device scripts (run on the Jetson, not Colab)

The following are executed on the Jetson Orin Nano after transferring the ONNX artefacts and GGUF checkpoints from Drive.

Jetson 05-1 | Build the YOLO-World TensorRT engine (device-specific)

```
# Run ON the Jetson. Engines cannot be cross-compiled in Colab.
/usr/src/tensorrt/bin/trtexec \
  --onnx=yoloworld_640.onnx \
```

```
--saveEngine=yoloworld_640_fp16.engine \
--fp16 \
--memPoolSize=workspace:2048
```

Jetson 05-2 | Launch Gemma 4 E2B with vision via native llama.cpp

```
# Native llama.cpp build (CUDA arch 87). The pre-built NVIDIA image is
# text-only; the native build is required for image input.
./llama-server \
--model gemma-4-E2B-it-Q4_K_M.gguf \
--mmproj mmproj-gemma4-e2b-f16.gguf \
-ngl 99 --flash-attn on \
--ctx-size 2048 \
--no-mmproj-offload \
--port 8080
```

Jetson 05-3 | Grounding tool wrapper (YOLO-World via TensorRT)

```
import tensorrt as trt, numpy as np
# Pseudocode-level wrapper: load engine, accept class name, return boxes.
class Grounder:
    def __init__(self, engine_path):
        self.engine = self._load(engine_path) # TensorRT runtime
    def find_object(self, name, frame, depth_map):
        boxes = self._infer(frame, classes=[name]) # ~160 ms
        if not boxes:
            return None
        x1,y1,x2,y2 = boxes[0]
        cx, cy = int((x1+x2)/2), int((y1+y2)/2)
        dist = float(np.median(depth_map[cy-5:cy+5, cx-5:cx+5]))
        direction = 'left' if cx < frame.shape[1]*0.35 else \
                    'right' if cx > frame.shape[1]*0.65 else 'center'
        return {"name": name, "distance_m": round(dist,2),
                "direction": direction}
```

Jetson 05-4 | Main loop (two-speed orchestration)

```
# Fast loop runs in its own thread, always on, non-ML.
# Slow loop is triggered by the wake word.
def on_user_query(audio):
    query = stt.transcribe(audio) # faster-whisper
    frame = camera.get_wide_frame()
    frame = privacy_gate(frame) # MANDATORY
    reply = gemma.chat(system=ARGUS_SYSTEM,
                       image=frame, text=query,
                       tools=[TOOL_SPEC]) # Gemma 4 E2B
    if reply.tool_call == "find_object":
        result = grounder.find_object(
            reply.args["name"], frame, depth.latest())
        reply = gemma.chat(tool_result=result) # compose final answer
    tts.speak(reply.text) # Piper
```

8. On-Device Deployment and Memory Budget

With YOLO-World replacing the 3B grounder, the memory budget is comfortable rather than marginal. The grounder is now a few hundred megabytes rather than several gigabytes, and no model swapping is required: every component is resident simultaneously.

Component	Precision	Approx. memory	Residency
Gemma 4 E2B	INT4 (GGUF)	~2.0–2.5 GB	Resident
YOLO-World	FP16/INT8 (TensorRT)	~0.3–0.5 GB	Resident
Stereo depth (RAFT)	INT8 (TensorRT)	~0.2 GB	Resident
SLAM (map + state)	—	~0.5 GB	Resident

Component	Precision	Approx. memory	Residency
Privacy gate (SCRFD + CRAFT)	INT8	~0.2 GB	On demand
Speech (Whisper, Piper)	INT8 (CPU)	~0.1 GB	CPU
OS, CUDA context, buffers	—	~1.5 GB	Resident
Total	—	~4.8–5.5 GB	Within 8 GB, with headroom

Consequence of the substitution

Because the grounder no longer competes with the agent for memory, the on-demand model-swapping contingency described in the previous plan is no longer required. All models remain resident, eliminating swap latency and simplifying the runtime.

9. Phased Execution Roadmap

Phase	Description	Resource	Duration
0	Feasibility verification	—	Complete
1	YOLO-World validation, fine-tune, ONNX export	T4 / L4	1–2 weeks
2	Privacy gate integration	T4	1 week
3	Gemma 4 agent + tool contract (Jetson)	Jetson	2 weeks
4	Stereo depth + safety reflex + SLAM	Local / Jetson	2 weeks
5	Speech pipeline integration	Jetson	1 week
6	Full on-device integration	Jetson	2–3 weeks
7	HCI study with participants	—	4–6 weeks

Phase 0 is now closed. The critical risk it addressed has been resolved by the grounder substitution rather than left open, which materially de-risks the remainder of the project.

10. Risk Register

Risk	Likelihood	Impact	Mitigation
YOLO-World open-vocabulary accuracy insufficient for rare objects	Medium	Medium	Light fine-tune on indoor objects; retain NanoOWL as alternative
Gemma 4 latency uncomfortable for interaction	Low	Medium	E2B (not E4B); cap context at 2048 tokens; cache recent scene
Privacy gate bypassed in integration	Low	High	Enforced as a hard precondition of every agent call; verified in testing
TensorRT engine build issues on Jetson	Medium	Low	Build on device per documented trtexec recipe; FP16 fallback if INT8 calibration is troublesome
Safety reflex inadequate between agent calls	Low	High	Fast loop remains fully non-ML, deterministic, continuously running

11. Summary of Changes

Aspect	Previous (v2) plan	This (final) plan
Grounding tool	LocateAnything-3B (function-called)	YOLO-World via TensorRT (function-called)
Grounder deployability	Identified as the key open risk	Resolved: LocateAnything not deployable on Orin Nano; YOLO-World substituted
Grounding latency	Several seconds (projected)	~160 ms per query (measured class)
Memory budget	~6.5–7.0 GB worst case, swapping contingency	~4.8–5.5 GB, all resident, no swapping
Reasoning agent	Gemma 4 E2B	Gemma 4 E2B (unchanged, now confirmed)
Architecture	Two-Speed	Two-Speed (unchanged)
Privacy gate	Mandatory	Mandatory (unchanged)

In summary, the Phase-0 investigation removed the principal uncertainty in the project by establishing that the reasoning agent is deployable as planned while the originally selected grounder is not. Substituting YOLO-World preserves every capability promised in the original proposal — including open-vocabulary object finding — while improving grounding latency by roughly an order of magnitude, relaxing the memory budget, and eliminating the need for model swapping. The architecture, the safety design, and the privacy obligations are all retained unchanged.